MySQL

Rev 6.4 del 16/04/2025

Sintassi base di **MySQL** 2
Query di selezione 2
Tipi di dati 2

I Comandi CRUD (DML) 4

Approfondimenti sulle query di selezione 6
Creazione e Cancellazione di una tabella (DDL) 8
ENUM e SET 9
DATA e ORA 9
Indici 10

Principali Operatori e Funzioni di MySQL 11

Configurazione per l'accesso attraverso la rete 13

Sintassi Base di MySQL

Caratteristiche

DBMS **client-server** veloce, multiutente, affidabile, estrema sicurezza con possibilità di gestire le SSL Secure Cockets Layer per proteggere i dati durante il passaggio tra server e client. Gestisce una elaborazione **multi-threading**, che permette a più client di connettersi contemporaneamente al database e di eseguire contemporaneamente interrogazioni diverse. Versione attuale **11.2** con php version **8.5** (aprile 2025)

Usa un linguaggio SQL conforme allo standard ANSI SQL92.

Formati di memorizzazione delle informazioni

Il formato **MyISAM** utilizzato inizialmente per la gestione delle tabelle non gestiva i comandi di transazione COMMIT e ROLLBACK. Dalla versione 5 in avanti è stato incluso un nuovo gestore denominato **InnoDB** in grado di gestire le transazioni con i comandi di COMMIT e ROLLBACK. L'attributo ***TYPE=InnoDB*** di CREATE TABLE consente di creare tabelle nel formato InnoDB anziché nel formato standard MyISAM.
Riguardo alla *codifica dei caratteri* il formato normalmente utilizzato era **`utf8mb4_unicode`** (unicode standard) o **`utf8mb4_general`** che semplifica alcune regole di confronto/ordinamento a vantaggio di una maggiore velocità.
`utf8mb4` è l'evoluzione di **`utf8`** e salva gli unicode su 4 byte contro i 3 bytes del formato precedente.
Dal 2020 è raccomandato l'utilizzo del set **`utf8mb4_0900_ai_ci`** basato sullo standard unicode 9.0

Tipi di Dati

Attributi utilizzabili per tutti i tipi di campi:

NULL Indica che nel campo sono ammessi valori NULL,

DEFAULT Indica il valore da assegnare per default al campo. Se su un campo di tipo stringa si specifica NULL (o viene omesso), il NULL viene automaticamente convertito in stringa vuota, ed il record viene inserito correttamente anche in assenza del check NULL. Affinché questa conversione **non** venga eseguita in automatico, occorre eseguire il seguente comando di configurazione:

```
ALTER TABLE canzoni ADD CONSTRAINT chkTitoloNonVuoto CHECK (Titolo<>'');
```

I Dati Numerici

- **TINYINT** – 1 byte - Valore numerico compreso tra –128 e 127. TINYINT UNSIGNED 0 e 255
- **SMALLINT** – 2 byte - Valore numerico tra –32768 e 32767. SMALLINT UNSIGNED 0 e 65535
- **MEDIUMINT** – 3 byte Valore numerico tra –8.388.608 e 8.388.607. MEDIUM UNSIGNED 0 e 16.777.215
- **INT** o **INTEGER** – 4 byte Valore numerico tra –2.147.483.648 e 2.147.483.647. INT UNSIGNED 0 e 4Mld
- **BIGINT** – 8 byte Valore numerico compreso tra 6 2^{63} BIGINT UNSIGNED 0 e 2^{64}
- **FLOAT** (m,d) – 4 byte - Floating point tra -3.402823466E+38 e -1.175494351E-38.
- **DOUBLE**(m,d) o **REAL**(m,d) – 8 byte - Floating point tra -1.7976931348623157E+308 e -2.2250738585072014E-308
- **DECIMAL** (m,d) o **NUMERIC**(m,d) – Valore numerico con decimali memorizzato come serie di caratteri.

Attributi per i dati numerici:

UNSIGNED numero senza segno. Per tutti il default è SIGNED, che può essere omesso.

ZEROFILL riempie con zeri l'ampiezza indicata.
Es N INT (5) ZEROFILL . Se N = 46 verrà visualizzato come 00046.
N può comunque contenere numeri a 6, 7 cifre.
Es N FLOAT (6,3) ZEROFILL (6 chr complessivi, compresa la virgola e 3 chr dopo la virgola).
Se N = 3,58 verrà visualizzato come 03,580

AUTO_INCREMENT Per tutti i tipi interi si può specificare l'attributo AUTO_INCREMENT che trasforma il campo in un contatore. In ogni tabella è ammesso un solo campo AUTO_INCREMENT che deve necessariamente essere NOT NULL e deve avere associato un indice (PRIMARY KEY o UNIQUE).

Come in Access, i valori utilizzati da un contatore non sono riutilizzabili, tranne nel caso in cui si svuoti completamente la tabella. L' AUTO_INCREMENT parte normalmente da 1, ma si può forzare un altro valore:

```
CREATE TABLE Libri
( CodiceLibro ID INT AUTO_INCREMENT NOT NULL PRIMARY KEY, titolo CHAR(30) )
AUTO_INCREMENT = 1000      // i codici partono da 1000
// Il campo AUTO_INCREMENT deve essere introdotto come NULL o essere tralasciato.
INSERT INTO Libri VALUES (NULL,  "Il fu Mattia Pascal")
```

Le Stringhe

- **CHAR** (M) – Stringa a lunghezza fissa pari a M (max 255). Il campo viene completato con degli spazi. Quando si legge il contenuto del campo, gli spazi finali vengono rimossi, per cui è come se non ci fossero. CHAR senza dimensione equivale a CHAR(1)
- **VARCHAR** (M) – Rispetto a CHAR vengono memorizzati soltanto i caratteri effettivamente presenti (max **255**) più un byte per la lunghezza. Eventuali caratteri eccedenti vengono eliminati.
  I campi definiti CHAR con lunghezza > 4 vengono automaticamente convertiti in VARCHAR

  L'attributo **BINARY**, utilizzabile sui campi CHAR e VARCHAR, trasforma il campo in *case sensitive*. Normalmente le stringhe sono gestite come *case unsensitive*
- **TINYTEXT** – Lunghezza variabile senza il parametro M. Max 255 caratteri + 1 byte per la lunghezza
- **TEXT** – Lunghezza variabile. Max **65.535** caratteri + 2 byte per la lunghezza
- **MEDIUMTEXT** - Lunghezza variabile. Max 16.777.215 + 3 byte per la lunghezza
- **LONGTEXT** - Lunghezza variabile. Max 4.294.967.295 + 4 byte per la lunghezza
- **TINYBLOB BLOB MEDIUMBLOB LONGBLOB** – Come TEXT ma case sensitive (BLOB = Binary Large Object)
- **ENUM** ("Francia","Germania",…) – Campo enumerativo simile a quello di Access. Max 65.565 voci
- **SET** ("Francia","Germania",…) – Campo enumerativo a valori multipli Max 64 voci

Query di Selezione

```
SELECT  ID, nome                  // campi da visualizzare tramite selezione verticale
FROM  studenti                    // nome della tabella da utilizzare per la ricerca
WHERE residenza = "fossano"       // filtro orizzontale su quali record visualizzare
   OR residenza = "cussanio"
```

Note sulle Query di selezione

- Le istruzioni SQL sono completamente **case unsensitive**. Sono infatti case unsensitive :
  - **I comandi SQL** (select, from, where, etc)
  - **I nomi dei campi**
  - **Il contenuto delle stringhe passate al where**
- Sono invece **case sensitive** le **chiavi** ed i **valori** del vettore restituito al termine della query

I nomi dei campi possono facoltativamente essere racchiusi all'interno di un **`back tick`** che quindi non può essere usato come delimitatore di stringa, nemmeno in fase di scrittura di una query (nel qual caso MYSQL interpreterebbe la stringa come nome di campo !).

Nel caso di query che ritornano **campi con lo stesso nome, mySQL** non ritorna MAI nomi composti (filiali.id), ma **l'ultimo campo con lo stesso nome maschera i precedenti**. E' possibile procedere in due modi:

1. Estrarre dalle tabelle secondarie **solo** i campi interessati che abbiano comunque nomi univoci:
   `SELECT concerti.*, citta.nomeCitta, generi.nomeGenere`
2. Utilizzare degli ALIAS nel caso in cui sia necessario restituire un campo avente lo stesso nome in 2 tabelle:
   `SELECT concerti.*, citta.nome AS nomeCitta, generi.nome AS nomeGenere`

I comandi CRUD (DML)

INSERT INTO (POST)

Consente di aggiungere nuovi record al fondo di una tabella esistente

```
INSERT INTO libri (Autore, `Titolo`)  // i campi possono essere scritti CON o SENZA backtick
 VALUES
  ('Pirandello', 'Il fu Mattia Pascal'),
  ('Wilde", "$myTitle");
```

Nel caso in cui si passino tutti i campi, si possono omettere i nomi dei campi e passare tutti i valori nello stesso ordine in cui i campi sono definiti all'interno del database. Ai campi di AutoIncrement occorre assegnare il valore NULL. Idem per i campi da lasciare vuoti. Occorre però assolutamente specificare un valore per ogni campo esistente.

```
INSERT INTO libri
 VALUES
  ( 101,  'Pirandello', 'Il fu Mattia Pascal', 7.30, 0) ,
  ( 102,  'Pirandello', 'Uno Nessuno Centomila', 7.30, 0) ,
  ( NULL,  'Wilde', 'Il ritratto di Dorian Gray', 7.30, 0);
```

Anziché indicare valori diretti, è possibile leggerli da un'altra tabella mediante una istruzione SELECT che deve restituire tipi corrispondenti a quelli attesi da Insert Into

```
INSERT INTO Libri (CodiceLibro, Autore, Titolo)
SELECT CodiceLibro, Autore, Titolo
FROM LibriOld
WHERE Prezzo Is Not Null;
```

Il tipo JSON

Il tipo **JSON** è stato introdotto in MariaDB 10.2.7 come alias di LONGTEXT per ragioni di compatibilità con il tipo JSON di MySQL. Accetta come valore una stringa JSON valida e opportunamente serializzata.

In fase di creazione non si specifica la lunghezza che, per un LONGTEXT, può arrivare fino a 4 MB.

Inserimento manuale di un nuovo record

```
CREATE TABLE `db`.`candidati`
( `id` INT UNSIGNED NOT NULL AUTO_INCREMENT ,
  `nome` VARCHAR(50) NOT NULL ,
  `esperienze` JSON NOT NULL ,
   PRIMARY KEY (`id`)
)

INSERT INTO `candidati` (nome, esperienze)
VALUES( 'Sergio',
  '[ { "incarico":"programmatore",
       "settore":"Java",
       "inizio":2008,
       "fine":2013,
       "azienda":"ABC s.r.l."
     },
     { ..........   }
   ]'
)
```

REPLACE (PUT)

Molto simile a INSERT INTO, con la stessa sintassi. L'unica differenza è che se si cerca di inserire un record avente chiave primaria già esistente (o chiave UNIQUE già esistente), mentre INSERT INTO segnala un errore, **REPLACE** cancella il vecchio record sostituendolo con quello nuovo.

UPDATE (PATCH)

Consente di modificare selettivamente i singoli campi di uno o più record

```
UPDATE Libri
SET Prezzo = Prezzo * 1.1,
titolo = UCASE(titolo)
  WHERE CasaEditrice = 'Einaudi';  // facoltativo
```

All'interno di SET si può utilizzare Null per azzerare un campo: SET Prezzo = Null;

DELETE

La sintassi del comando DELETE è la stessa identica del comando SELECT. DELETE provvede a cancellare i record estratti. A differenza di Access, dopo DELETE **occorre omettere** l' *.

```
DELETE  FROM Libri WHERE Prezzo Is Null;
```

Se non si specifica il WHERE vengono rimossi tutti i record della tabella.

Approfondimento sulle Query di selezione

SELECT [DISTINCT] elenco_campi
[**FROM** elenco_tabelle]
[**WHERE** filtro_recordset [AND |OR | BETWEEN | LIKE | IN | IS NULL]]
[GROUP BY nome_colonna]
[HAVING filtro_sui_gruppi]
[ORDER BY nome_colonna [ASC | DESC]]
[LIMIT [posizione,] numero_righe];

DISTINCT fa in modo che vengano restituite soltanto righe *univoche*
LIMIT è la clausola SQL standard equivalente al predicato TOP di Access.
Posizione indica il numero di record da ignorare prima di iniziare la visualizzazione.

Nello standard SQL anche la clausola FROM è opzionale. E' possibile utilizzare l'istruzione SELECT semplicemente per valutare un'espressione o il risultato di una funzione : `SELECT NOW( );`

JOIN fra due o più tabelle

Nel caso di una relazione 1:N join restituisce la tabella N 'estesa', cioè restituisce tutti i record della tabella N aggiungendo a fianco di ogni record tutte le informazioni provenienti dalla tabella 1. Ad esempio nel caso di Nazioni (1) e Piloti (N) restituisce l'elenco di tutti i piloti con fianco il nome anagrafico della nazione.

Sono supportate sia la sintassi SQL standard, che definisce la condizione di join all'interno della clausola where,

SELECT Piloti.Nome
FROM Piloti, Nazioni
WHERE Piloti.CodiceNazione = Nazioni.CodiceNazione AND Nazioni.Nome = 'Italia';

sia la sintassi più recente INNER JOIN ON che consente anche gli outer join ***(ed anche più intiutiva)***

SELECT Piloti.Nome
*FROM Piloti **INNER JOIN** Nazioni **ON** Piloti.CodiceNazione = Nazioni.CodiceNazione*
WHERE Nazioni.Nome = 'Italia';

Il **Left Join** (outer join) restituisce tutti i record della tabella di sinistra.
Il **Right Join** (outer join) restituisce tutti i record della tabella di destra.

La funzione count()

L'asterisco utilizzato con count chiede alla query di considerare tutte le righe esistenti nel gruppo e di contarle. In questo modo non si dà importanza al valore delle righe, ma solo al fatto che siano presenti o meno. Access non supporta altre opzioni. Lo standard SQL consente invece di specificare all'interno delle parentesi di COUNT il nome di un campo. In tal caso il conteggio riguarderà soltanto le righe in cui quel campo presenta un valore diverso da NULL. Inoltre all'interno delle parentesi di COUNT è possibile utilizzare anche il predicato DISTINCT, in modo da conteggiare soltanto quei record che presentano valore differente sul campo indicato

```
SELECT COUNT(DISTINCT campo) FROM tabella
```

SELECT INTO OUTFILE

Il SELECT produce una tabella virtuale che non viene memorizzata su disco. SELECT INTO consente di memorizzare il risultato all'interno di un file di testo. La sintassi è la stessa di LOAD DATA INFILE

```
SELECT *
  INTO OUTFILE 'FileDati.txt'
  FIELDS TERMINATED BY ';'   ENCLOSED BY '\"'
  LINES TERMINATED BY '\n'
FROM libri;
```

Operatore IN

Le query annidate non sono consentite. L'operatore IN consente soltanto la scelta fra un certo numero di valori indicati direttamente:

```
SELECT * FROM tabella WHERE campo IN ('valore1', 'valore2');
```

Query combinate mediante l'operatore UNION

Consente di combinare tra loro i recordset prodotti da 2 query indipendenti, recordset che devono essere chiaramente dello stesso tipo. L'operatore UNION che dispone dei due predicati ALL , che aggiunge indistintamente tutti i record del secondo recordset in coda al primo recordset, e DISTINCT, che aggiunge in coda solo i record del secondo recordset che non hanno corrispondenza all'interno del primo recordset.

Query combinate mediante l'operatore INTERSECT

Se connettiamo due comandi SELECT con un operatore INTERSECT, si ottengono soltanto quei record che sono presenti in entrambi i recordset in questione. Esempio:

```
SELECT * FROM tabella WHERE a = 3
INTERSECT
  SELECT * FROM tabella2 WHERE b = 5;
```

Query combinate mediante l'operatore MINUS

Se connettiamo due comandi SELECT con un operatore MINUS, si ottengono soltanto quei record che sono presenti nel primo recordset, ma non nel secondo Esempio:

```
SELECT * FROM tabella WHERE a = 3
MINUS
  SELECT * FROM tabella2 WHERE b = 5;
```

Motori di ricerca: MATCH AGAINST

Funzionalità non standard introdotta a partire dalla versione 4.0. Si supponga di dover trovare un titolo di cui non si ricorda esattamente il nome. Per poter utilizzare MATCH AGAINST occorre innanzitutto indicizzare il campo Titolo mediante un indice FULLTEXT : `ALTER TABLE libri ADD FULLTEXT (Titolo);`

SELECT titolo FROM libri
WHERE MATCH (titolo) AGAINST ('Mattia Pascal');

L'algoritmo utilizzato per ricercare le corrispondenze è il cosiddetto algoritmo *fuzzy* che ordina i record restituiti in base al numero di parole della frase "Mattia Pascal" che hanno corrispondenza all'interno del campo titolo, e, a parità di ricorrenze, porta in alto quelle in cui la ricorrenza costituisce la parte proporzionalmente maggiore rispetto alla lunghezza del campo. Il confronto è chiaramente *case unsensitive*. Inoltre non ricerca:

- Le parole di lunghezza minore uguale a 3 caratteri
- Le parole "comuni" sulla base di una apposita stop-word-list
- La parole che compaiono in più del 50% delle righe della tabella

E' possibile anche utilizzare gli operatori + e – per ricercare solo quei record che hanno match su entrambi i nomi oppure quei record che hanno match su un nome ma non sull'altro.

SELECT titolo FROM libri
WHERE MATCH (titolo) AGAINST ('+Mattia +Pascal' IN BOOLEAN MODE);
WHERE MATCH (titolo) AGAINST ('-Mattia +Pascal' IN BOOLEAN MODE); // Pascal ma non Mattia

Creazione e cancellazione di una tabella

```
CREATE TABLE [IF NOT EXISTS] libri
  ( CodiceLibro INT NOT NULL  PRIMARY KEY,
    Autore TEXT NOT NULL,
    Titolo TEXT NOT NULL,
    Prezzo FLOAT,
    Disponibile TINYINT DEFAULT  1);     // Non esiste il Boolean
```

Si può andare a capo in qualsiasi momento. Andando a capo MySQL utilizza il prompt -> che significa che il comando non è ancora terminato ed il sistema è in attesa di continuazione del comando. In corrispondenza del punto e virgola il comando sarà considerato chiuso e verrà inviato in esecuzione.

DROP TABLE libri; // Cancella la tabella libri e tutti i dati contenuti

SHOW TABLES; // Mostra l'elenco delle tabelle contenute nel database corrente

DESCRIBE Libri; // Mostra la struttura della tabella Libri (Tipo, Not Null, Indici, Default). Ottimo.

ALTER TABLE libri // Modifica la struttura della tabella libri
ADD [COLUMN] CasaEditrice TEXT [FIRST | AFTER NomeColonna],
DROP [COLUMN] Prezzo,
MODIFY [COLUMN] Titolo VARCHAR(100),
CHANGE [COLUMN] Autore Scrittore VARCHAR(100), // Cambia nome e tipo
ALTER [COLUMN] Disponibile { SET DEFAULT 2 | DROP DEFAULT },
RENAME NuovoNomeTabella,
ADD PRIMARY KEY (CodiceLibro), // Trasforma CodiceLibro in chiave primaria, se ancora non lo era
ADD INDEX IndiceAutore (Autore), // Crea un IndiceAutore non univoco sul campo Autore
ADD UNIQUE IndiceTitolo (Titolo), // Crea un IndiceTitolo univoco (chiave) sul campo Titolo
DROP PRIMARY KEY
DROP INDEX IndiceAutore
DROP UNIQUE IndiceTitolo;

SHOW INDEXES FROM libri; // Mostra l'elenco degli indici della tabella Libri

SHOW KEYS FROM libri; // Analogo.

Creazione di tabelle temporanee

Come detto MySQL non supporta le interrogazioni annidate. Supporta però una interessante variante dell'istruzione SQL standard SELECT INTO che consente di memorizzare il risultato di una query all'interno di una tabella temporanea che verrà rimossa al termine della connessione corrente al database.

```
CREATE TEMPORARY TABLE  tmp
  SELECT codicelibro As codice, autore, titolo
  FROM libri
  WHERE casaeditrice = "einaudi";
```

La tabella temporanea tmp potrà essere utilizzata nelle query al pari di qualunque altra tabella reale.

Creazione e cancellazione di un database

SHOW DATABASES; // Mostra l'elenco dei database esistenti

CREATE DATABASE [IF NOT EXISTS] biblioteca; //Crea la cartella corrispondente nel percorso /Data

DROP DATABASE biblioteca; // Cancella tutte le tabelle e tutti i dati contenuti

ENUM

```
CREATE TABLE Nazioni (
  Nome ENUM ("Italia", "Francia", "Germania")  NULL )
```

NULL indica che il campo accetta i valori NULL. Il default è NOT NULL.
Alla voce "Italia" viene automaticamente associato l'indice 1, alla voce "Francia" l'indice 2 e così via.
Al valore NULL viene associato l'indice NULL. Qualunque voce non compresa fra quelle impostate viene comunque memorizzata nel database con indice 0 (compresa la stringa vuota).

Supponendo di partire dalla tabella creata al punto precedente, il seguente comando

INSERT INTO Nazioni VALUES ("Germania"), ("Italia"), (NULL), ("Spagna"), (2), ("Italia")

Inserisce nel database **7 record** aventi i seguenti indici: 3, 0, 1, NULL, 0, 2, 1

Quando si inserisce un valore numerico, questo viene automaticamente considerato come un indice.

Per accedere agli indici da SQL si utilizza il nome del campo seguito da +0.
SELECT Nome+0, Nome FROM Nazioni // Visualizzo gli indici con a fianco i nomi delle nazioni.

Un eventuale ORDER BY esegue un ordinamento sugli indici: NULL, 0, 1, 2, 3
L'ENUM si presta bene per la costruzione di un Boolean : 1=False, 2=True

SET

a differenza di ENUM, consente di inserire contemporaneamente all'interno del campo più valori enumerativi, al limite anche tutti.

CREATE TABLE Nazioni (
Nome SET ("Italia", "Francia", "Germania") NULL)

INSERT INTO Nazioni VALUES ("Germania"), ("), ("Italia, Francia"), (NULL), ("Spagna")

Inserisce nel database 5 record aventi i seguenti indici: 3, 0, 12, NULL, 0
dove il 12 realtà è un 1 concatenato ad un 2

DATA e ORA

- DATE – Data in formato "AAAA-MM-GG" Come separatore è ammesso ogni tipo di punteggiatura
  Il tipo DATE è su 3 byte e va dal 01/01/1000 al 31/12/9999
- TIME – Ora nel formato: "hh:mm:ss" Come separatore è ammesso ogni tipo di punteggiatura
- DATETIME – Data e Ora, in formato 'AAAA-MM-GG hh:mm:ss'
- YEAR – Anno nel formato "AAAA" . Occupa un solo byte

Se si cerca di immettere in un campo DATE o TIME una informazione non valida, questa viene memorizzata come 00:00:00. Purtroppo è possibile confondere questo valore di data non valida con l'ora valida 00:00:00. E' l'applicazione client che dovrà preoccuparsi di non immettere dati non validi.

E' possibile copiare direttamente dati da un tipo ad un altro. Se copiamo ad esempio un DATETIME in un DATE l'informazione relativa all'ora viene rimossa. Se copiamo un DATE in un DATETIME l'ora viene settata a 00:00:00

INDICI

Velocizzano le ricerche, perché le ricerche stesse vengono eseguite su tabelle ordinate.
Se l'indice è univoco, le prestazioni sono ancora migliori perché il DBMS sa che c'è un solo record da recuperare.
E' bene creare un indice per tutti quei campi su cui probabilmente verranno eseguite ricerche frequenti.
Tutte le chiavi esterne devono essere sempre indicizzate.
E' del tutto inutile indicizzare campi che contengono al loro interno pochi valori (ad esempio il campo genere M/F)
E' inutile indicizzare campi su tabelle contenenti pochi record, sui quali si può eseguire una scansione sequenziale.

Un indice può essere applicato a un singolo campo oppure a campi multipli.

MySQL supporta 4 tipi di indici:

PRIMARY KEY (UNIQUE + NOT NULL)
UNIQUE
INDEX o KEY
FULLTEXT

Su ciascuna tabella è ammesso un solo indice di tipo PRIMERY KEY (che è un indice univoco su un campo *necessariamente* definito NOT NULL). INDEX | KEY è un indice generico non univoco. Gli indici UNIQUE e INDEX accettano il valore NULL all'interno del campo. L'indice FULLTEXT serve per eseguire ricerche con le istruzioni MATCH AGAINST. Per ogni tabella è ammesso un limite massimo di 16 indici.

Gli indici possono essere definiti al momento del CREATE TABLE oppure aggiunti / rimossi in un secondo tempo mediante le istruzioni ALTER TABLE ADD / DROP. All'interno dell'istruzione CREATE TABLE gli indici vengono dichiarati *al termine* della dichiarazione dai campi, mediante le seguente sintassi:

TIPO [NomeIndice] (Elenco_colonne)

Se NomeIndice non viene specificato, l'indice assume automaticamente lo stesso nome del campo.

```
……,
PRIMARY KEY (CodiceLibro),
INDEX (Autore),
UNIQUE IndiceTitolo (Titolo, CasaEditrice);
```

Il PRIMARY KEY può anche essere specificato a fianco del campo.
Gli indici a campi multipli vanno bene quando si pensa di dover eseguire spesso delle interrogazioni che agiscono contemporaneamente su quei campi:

```
WHERE Titolo = "PIPPO" AND CasaEditrice = "PLUTO"
```

Per aggiungere l'indice in un secondo tempo, oltre alle già viste istruzioni ALTER TABLE ADD / DROP INDEX, MySQL supporta anche la sintassi *non* standard CREATE PRIMARY KEY, CREATE UNIQUE, CREATE INDEX utilizzata da molti database fra cui Access.

Gli indici non sono applicabili ai campi di tipo TEXT, ma solo sui campi di tipo CHAR e VARCHAR.
Per i campi TEXT è riconosciuto soltanto l'indice FULLTEXT.

Principali Operatori e Funzioni di MySQL

ASCII	Restituisce il valore ASCII di un carattere. Sintassi: ASCII('a');
CONCAT	Concatena due stringhe. Sintassi: CONCAT (stringa1, stringa2);
LENGTH	Restituisce la lunghezza di una stringa. Sintassi: LENGHT(stringa); Esempio: SELECT LENGHT('prova'); restituirà 5. Se per esempio vogliamo selezionare tutti i record che hanno Campo1 di lunghezza maggiore o uguale a 3 faremo: SELECT (*) FROM tabella WHERE LENGHT(Campo1) >= 3;
LIKE	E' una condizione del WHERE. Seleziona dei dati da un database dove il campo è simile ad uno da noi specificato. Il carattere jolly per indicare infiniti caratteri è il simbolo di percento (%), mentre il carattere jolly per indicare un singolo carattere è l'underscore (_). Il primo esempio troverà ad esempio panettone, panettiere, appannato, mentre il secondo esempio troverà pala, para, paca, etc… Primo esempio: SELECT * FROM tabella WHERE campo LIKE '%pan%'; Secondo esempio: SELECT * FROM tabella WHERE campo LIKE 'pa_a';
LOWER	Riduce in minuscolo tutti i caratteri di una stringa. Sintassi: LOWER(stringa); Esempio: SELECT LOWER('PROVA'); restituirà: prova.
UPPER	Trasforma in maiuscolo tutti i caratteri di una stringa. Sintassi: UPPER(stringa); Esempio: SELECT UPPER('prova); restituirà: PROVA.
LEFT	Restituisce una parte della stringa partendo da sinistra. Sintassi: LEFT(stringa,numero_caratteri); Esempio: SELECT LEFT('prova',3); restituirà 'pro'.
NOW	Restituisce data e ora corrente. Se il risultato viene copiato in un campo DATE viene mantenuta soltanto l'informazione relativa alla data. Se viene copiato in un campo TIME viene mantenuta sultanto l'informazione relativa all'ora.
RIGHT	Restituisce una parte della stringa partendo da destra. Sintassi: RIGHT(stringa,numero_caratteri); Esempio: SELECT RIGHT('prova',3); restituirà 'ova'.
SUBSTRING	Seleziona un determinato numero di caratteri da una stringa Sintassi: SUBSTRING(stringa,numero_carattere_d'inizio,numero_caratteri); Esempio: SELECT SUBSTRING('prova',2,3); restituirà 'rov'.
REPLACE	Sostituisce una serie di caratteri da una stringa con degli altri caratteri. Sintassi: REPLACE(stringa, carattere_da_trovare, sostituto); Esempio: REPLACE ('prova','ov', 'a'); Restituirà 'praa'.
REVERSE	Restituisce la stringa scritta al contrario. Sintasi: REPLACE(stringa); Esempio: SELECT REPLACE('stringa di prova'); Restituirà: 'avorp id agnirts'
TRIM	Elimina gli spazi a sinistra e a destra di una stringa. Sintassi: TRIM(stringa):
LTRIM	Elimina gli spazi a sinistra di una stringa. Sintassi: LTRIM(stringa)
RTRIM	Elimina gli spazi a destra di una stringa. Vedere anche LTRIM e TRIM. Sintassi: RTRIM(stringa):

RAND	Genera un numero casuale di tipo float compreso tra 0 e 1.
MOD	Restituisce il resto della divisione tra due valori. Sintassi: MOD(5,2); Esempio: SELECT MOD(5,2); darà come risultato 1.
ROUND	Restituisce il valore arrotondato con un certo numero di decimali. Sintassi: ROUND(valore, numero_decimali); Esempio: SELECT ROUND(3.444245,2); Restituirà 3.44 SELECT ROUND(3.5154654,2); Restituirà 3.52
FLOOR	Restituisce il valore integer maggiore che risulta minore o uguale all'espressione numerica specificata. Sintassi: FLOOR(valore); Esempio: `SELECT FLOOR(123.45), FLOOR(-123.45);` `Restituirà nell'ordine 123 e -124.`
CEILING	Restituisce il valore integer minore che risulta maggiore o uguale all'espressione numerica specificata. Sintassi: CEILING(valore); Esempio: `SELECT CEILING(123.45), CEILING($-123.45);` Restituirà 124.00 e -123.00
SQRT	Esegue la radice quadrata di un valore. Sintassi: SQRT(valore);
POWER	Restituisce un valore dato in input elevato ad un determinata potenza. Sintassi POWER(valore,potenza); Esempio: SELECT POWER(3,2); Restituisce 9.
SIN	Restituisce il seno di un valore espresso in radianti. Sintassi: SIN(valore);
COS	Funzione matematica che restituisce il coseno di un angolo espresso in radianti. Sintassi: COS(valore)
TAN	Restituisce la tangente di una espressione. Sintassi TAN(valore);
ATAN	Calcola l'arcotangente del valore tra parentesi; Sintassi: ATAN(valore);
COT	Funzione matematica che restituisce la cotangente di un angolo in radianti Sintassi: COT(valore);
LOG E **LOG10**	Restituisce il logaritmo di un numero (LOG10 lo stesso, ma in base 10). Sintassi: LOG(valore): LOG10(valore);
PI	Restituisce il valore di pigreco con 6 decimali. Sintassi: PI( );
RADIANS	Converte un valore da gradi in radianti. Sintassi: RADIANS(valore);

Configurazione per l'accesso attraverso la rete

Per consentire a MySql di accettare richieste provenienti da un'altra macchina della rete locale occorre :

1) Cliccare su **CONFIG** di MySql per aprire il file **my.ini**

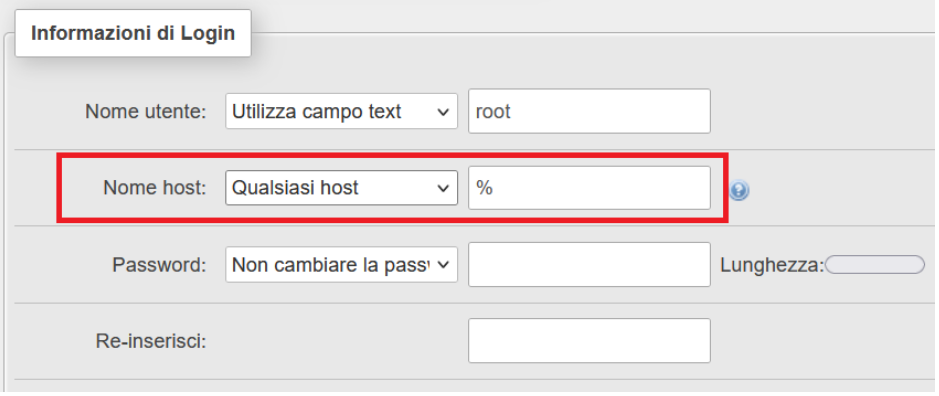
Aggiungere la seguente riga
`bind-address = 0.0.0.0`
Nel file è presente la seguente riga
`# bind-address="127.0.0.1"`
commentata perché rappresenta la condizione di default che è quella di consentire l'accesso soltanto dall'interfaccia di loopback 127.0.0.1
0.0.0.0 significa tutte le interfacce

2a) Modificare i privilegi dell'utente root

Da **MySql Admin** selezionare il DB corrente / **`Privilegi`**
Selezionare il primo utente root / **`Modifica Privilegi`**

	Nome utente	Nome host	Tipo	Privilegi	Grant	Azione	
☑	root	127.0.0.1	globale	ALL PRIVILEGES	Sì	Modifica privilegi	Esporta
☐	root	::1	globale	ALL PRIVILEGES	Sì	Modifica privilegi	Esporta
☐	root	localhost	globale	ALL PRIVILEGES	Sì	Modifica privilegi	Esporta

Nella finestra che compare è possibile impostare tutti i privilegi da concedere all'utente **`root`**
Selezionare **`Informazioni di Login`**
Impostare il Nome Host come in figura:



2b) In alternativa Creare un MySql un utente che possa connettersi da qualsiasi IP:

```
CREATE USER 'nomeutente'@'%' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON nomedatabase.* TO 'nomeutente'@'%';
FLUSH PRIVILEGES;
```

`'%'` indica che l'utente può connettersi da qualsiasi IP

Si potrebbe eventualmente specificare un singolo indirizzo IP valido:

```
CREATE USER 'nomeutente'@'192.168.1.100' IDENTIFIED BY 'password';
```

3) Impostare opportunamente lo script di accesso:

```
$host = "192.168.1.100";        // IP del server MySQL
$user = "nomeutente";
```